



[BIT703 Web Technologies](#) / [Contents](#) / Topic 7: Client-side web application development and programming – JavaScript / Basics of JavaScript

JavaScript loops 20m

A loop will repeat a block of code until a condition is met. Loops are useful to use if you want to run the same code several times, each time with a different value. You often use loops when you use arrays. There are 'for' statements and 'for...of' statements.

Watch *Learn JavaScript for loops in 5 minutes!* (Bro Code, 2023) (5:44), to learn about for and for...of loops. This video provides a demonstration of how to use these loop statements.



ENTER FULLSCREEN

Skip to main content

Learn JavaScript FOR LOOPS in 5 minutes!



Learn JavaScript FOR LOOPS in 5 minutes!

Bro Code



Watch on

Search...



All right. What's going on people? So today I'm going to discuss for loops. A for loop will repeat some code a limited amount of times. With the while loop, a while loop can repeat some code infinitely. With for loop, we can do something a certain or limited amount of times.

Here's an example. Let's say we would like to display the word hello three times. With for loop, we can write something like this. Now to create a for loop type for parentheses curly braces. Within the set of parentheses, we have up to three statements. The first is that we can create a temporary counter much like we're assigning a variable.

Let's create a counter named i-- let i. i is a common naming convention for counters within a loop, i meaning index. So let i equal-- what number would we like to start at?

Let's start at 0, then console.log. This is the first statement. We'll create a counter to

Powered by 3PLAYMEDIA

[Skip to main content](#)

'for' loops

A 'for' loop statement creates a loop that runs a block of code a number of times.

The basic syntax for a 'for' loop is:

```
for ([counter start value]; [condition to end looping]; [iterator or  
counter update]) {  
  
  //loop over something  
  
}
```

'for' loops are often used to loop through an array.

For example, remember the array we used previously:

```
var fruitArray = ["apple", "banana", "pear"];
```

You could use the 'for' loop to write:

```
let text = "";  
  
for (let i = 0; i < fruit.length; i++) {  
  
  text += fruit[i] + " ";  
  
}
```

This will loop through the list of fruit items starting from index number 0 and end on the last fruit item because fruit.length returns the length of an array (so endpoint is the last point in the array).

See the output in the CodePen example below. You are welcome to add more fruits or to reverse the loop.

[Skip to main content](#)



HTML

JS

Result

EDIT ON

LIVE

```
<!DOCTYPE html>
<html>
<body>

<h2>For Loop Example</h2>

<p id="demo"></p>

</body>
</html>
```

Resources1×0.5×0.25×Rerun

Example 7.1 Loops



'for...of' loops

There is a variation of the 'for' statement which is the 'for...of' statement. This is used when iterating over variable collection types like arrays and strings. The syntax for a 'for...of' statement is.

```
for (variable of iterable) {  
  
  // code block to be executed  
  
}
```

Here variable means for every iteration the value of the next property is assigned to the variable. Variable can be declared with `const`, `let` or `var`, while iterable means an object that has iterable properties.

Inside the loop, the variable will contain the value of the next item in the data structure.

For example, using the array `var fruitArray = ["apple", "banana", "pear"]`; we could use the 'for...of' loop and write:

```
let text = "";  
  
for (let x of fruit) {  
  
  text += x + " ";  
  
}
```

This will iterate over the fruit array object and display all the values.



for...in loops

Another variation of the for loop is the 'for...in' loop statement, where we loop through the properties of an object. In other words, we can access the object elements without using an index. Like the 'for...of' loop, but with objects instead of iterables.

```
for (variable in object) {
```

Skip to main content

```
// code block to be executed  
  
}
```



For example, we can iterate over a JSON-like object such as in the following:



```
1  const customer = {firstname:"Joe", lastname:"Bloggs", age:25};  
2  
3  let text = "";  
4  for (let item in customer) {  
5      text += customer[item]; //item is firstname, lastname, age  
6  }
```



While loop

In the case of the while loop, we execute code while a condition is true. In other words, 'while' condition is 'true', perform the following code. This loop is very suitable for iterating over data structures where the initial size is unknown, like receiving JSON data from the server.



[Skip to main content](#)



```
1 let i = 0;
2 let sum = 0;
3 while ( i < 10) {
4     sum += i;
5     i++;
6 }
```



[Skip to main content](#)

Dig deeper

These resources are additional and are not required readings.

To explore loop statements more, read these resources below.

■ Resources from W3Schools:

- JavaScript for loop (W3Schools.com, n.d.)
- JavaScript for in loop (W3Schools.com, n.d.)
- JavaScript for of loop (W3Schools.com, n.d.)
- JavaScript while loop (W3Schools.com, n.d.)

■ Resources from Programiz:

- JavaScript for loop (Programiz, n.d.)
- JavaScript for...in loop (Programiz, n.d.)
- JavaScript for...of loop (Programiz, n.d.)
- JavaScript while and do...while loop. (Programiz, n.d.)



Thoughts on this page?

